POO

Programação Orientada a Objetos

Recapitulando...

- O que é uma classe?
- O que são atributos e métodos?

Introdução aos Pilares da POO

- Encapsulamento
- Herança
- Polimorfismo
- Abstração

Encapsulamento

Definição: consiste na ocultação dos dados internos de um objeto para proteger esses dados contra acessos ou modificações indevidas.

- Uso de atributos privados (__atributo)
- Métodos get e set

- ✅ Exemplo: Classe ContaBancaria com saldo privado.

Encapsulamento

Analogia simples:
Imagine um controle remoto de TV.
Você sabe quais botões apertar (volume, canal, ligar/desligar), mas não precisa saber como ele funciona por dentro.

O controle encapsula sua lógica interna.

Você interage com ele apenas por meio de uma interface clara e segura (os botões).

Exemplo Encapsulamento

```
class ContaBancaria:
    def __init__(self, titular, saldo):
        self.titular = titular
        self.__saldo = saldo

    def get_saldo(self):
        return self.__saldo

    def depositar(self, valor):
        if valor > 0:
            self.__saldo += valor
```

Herança

Definição: Permitir que uma classe herde atributos e métodos de outra.

Superclasse (pai)
Subclasse (filho)

✅ Exemplo: Animal → Cachorro, Gato.

Exemplo Herança

```
class Animal:
    def __init__(self, nome):
        self.nome = nome

    def falar(self):
        print("O animal faz um som.")

class Cachorro(Animal):
    def falar(self):
        print("Au au!")

dog = Cachorro("Rex")
dog.falar()
```

Exemplo Herança

Analogia simples:
Imagine que temos uma classe Animal com as características:

- nome
- método falar()

Agora criamos um Cachorro e um Gato — ambos são animais!
Logo, eles herdam tudo que Animal tem, mas também podem ter comportamentos próprios.

Métodos de Instância vs Classe

Métodos de instância: métodos comuns, que usamos o tempo todo. Eles dependem de um objeto (instância) para serem chamados, e usam o parâmetro self para acessar atributos e outros métodos da instância.

Métodos de classe: métodos que atuam na classe como um todo, e não dependem de uma instância específica.
Para criá-los, usamos o decorador @classmethod e o primeiro parâmetro é cls (representando a classe).

Métodos de Instância vs Classe

Métodos de instância: usam self → ligados a um objeto.
Métodos de classe: usam @classmethod e cls → ligados à classe.

```
class Pessoa:
    contador = 0

    def __init__(self, nome):
        self.nome = nome
        Pessoa.contador += 1

    @classmethod
    def total_pessoas(cls):
        return cls.contador
```